



A BEGINNER'S COURSE MANUAL

Building & Reading a Real App with Claude Code

How “Sparkle”, a cleaning planner for two, was designed, built, hosted and connected to a database — and how to read the code behind it with understanding, not guesswork.

Project: **Sparkle** — cleaning-scheduling · Repository: [Barcode-Bleeb/cleaning-scheduling](#)

Written as a companion to your first Claude Code project.

What's inside

- 1 How to use this manual (and the mindset to bring)
- 2 What Sparkle is — the whole system at a glance
- 3 From idea to app: the plan and how it grew
- 4 The three languages of the web
- 5 Anatomy of the app: reading `index.html`
- 6 Data & state: where the information lives
- 7 Following one action through the code
- 8 Making it an installable app (PWA, offline, updates)
- 9 Git & GitHub: version control and hosting
- 10 Supabase in depth: the backend that ties two phones together
- 11 How real-time sync actually works
- 12 Notifications & calendar feeds
- 13 Testing: how we proved it works
- 14 A little design literacy
- 15 Reading code like a reviewer: a beginner's rubric
- 16 Glossary
- 17 Where to go next

How to use this manual

The goal isn't to turn you into a professional programmer overnight. It's to make sure that when you look at your app's code, you recognise what you're seeing — so you're learning, not just trusting.

You built Sparkle by describing what you wanted and letting Claude Code write it. That's a powerful way to work, but it has a well-known trap nicknamed “**vibe coding**”: accepting code because it seems to work, without any sense of whether it's sound. This manual is the antidote. By the end you should be able to open `index.html`, scroll through it, and think things like “*this part draws the task list*,” “*this bit saves to storage*,” “*here's where it talks to the database*,” and even “*hmm, this looks like it could go wrong if the input is empty*.”

How the chapters are organised

- **Chapters 2–3** give you the big picture and the story of how the app grew — the “map” before the “streets.”
- **Chapters 4–7** teach you to read the actual code of the app.
- **Chapters 8–12** explain the moving parts around the code: installability, GitHub, Supabase, sync, notifications.
- **Chapters 13–15** are about judgement: how we proved the app works, a little design sense, and a practical rubric for spotting good vs. risky code.
- **Chapters 16–17** are a glossary and next steps.

KEY IDEA

You do not need to understand every character. You need to understand *shape and intent*: what a block of code is **for**, and whether it handles the obvious things that could go wrong. That skill is learnable, and it's what separates learning from vibe coding.

Three kinds of boxes you'll see

KEY IDEA

The thing worth remembering from a section.

WATCH OUT

A common mistake or a place where code often goes wrong — exactly the sort of thing you're learning to spot.

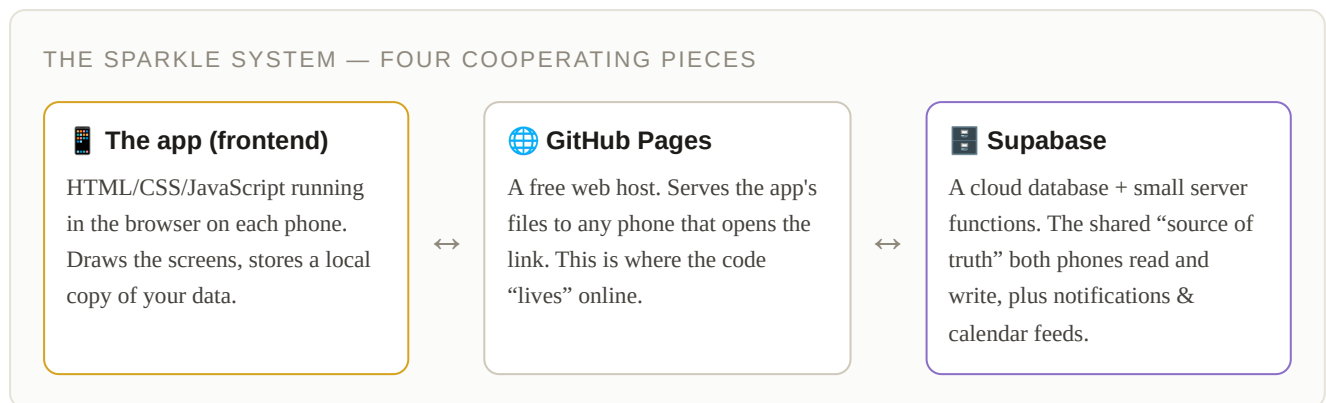
TRY IT

A small, safe experiment you can do yourself to make the idea concrete.

What Sparkle is — the whole system at a glance

Before any code, get the mental model. Sparkle is not one thing; it's a small set of cooperating pieces. Almost everything else in this manual is a zoom-in on one of them.

Sparkle is a **web app**: a program that runs inside a web browser rather than being installed from the App Store. On your iPhone it *looks* like a native app because you added it to your Home Screen, but under the hood it's a web page that happens to be very capable. This matters because it explains almost every design decision in the project.



Frontend vs. backend — the single most useful distinction

Almost all software is split into two halves, and naming them correctly will make everything clearer:

TERM	WHERE IT RUNS	IN SPARKLE
Frontend (client)	On the user's device (the phone's browser)	Everything in <code>index.html</code> : the task list, the buttons, the stats charts, the local data copy.
Backend (server)	On a computer in the cloud you don't see	Supabase: the shared database and the small “edge functions” that send notifications and serve the calendar feed.

A crucial consequence: **the frontend cannot be trusted with secrets or with being the single source of truth for two devices**. Anything the phone knows, a curious user could read. And two phones each running their own copy need something *in the middle* to agree on. That “middle” is the backend — which is exactly why Supabase entered the project.

What runs where, in one table

FILE	ROLE	RUNS ON
<code>index.html</code>	The entire app — screens + logic + styling in one file	Phone (browser)
<code>manifest.webmanifest</code>	Tells iOS “this is an installable app” (name, icon, colours)	Phone
<code>sw.js</code> (service worker)	Makes the app work offline and update itself	Phone (background)
<code>icons/</code>	The Home-Screen icons	Phone
<code>supabase/functions/...</code>	Small server programs: calendar feed & push notifications	Supabase (cloud)
Supabase database	The shared data both phones sync through	Supabase (cloud)

KEY IDEA

When something in the app confuses you, first ask: “*is this frontend or backend?*” Frontend = on the phone, in `index.html`. Backend = in Supabase. That one question resolves most confusion about “where does this happen?”

From idea to app: the plan and how it grew

Good software is rarely built all at once. It starts simple, is made to work, and then grows one deliberate step at a time. Your project is a textbook example — and the growth pattern is itself a lesson.

The original brief

The starting request was modest and clear: a tool for you and your partner to **log and plan a cleaning schedule**, with a handful of must-haves — plan tasks on a date, postpone a task, assign a task to a person, and a dashboard of statistics (who did what, how often, how many postponements, how often done on the first planned date). Crucially it should run on your iPhones.

KEY IDEA

Notice what the brief did *not* say: nothing about databases, accounts, notifications, or design systems. A good first version delivers the **core value** and deliberately leaves everything else out. This is called building a **minimum viable product (MVP)**.

The first version (the MVP)

The first build was intentionally the *simplest thing that could possibly work*: a single self-contained web page that stored all its data **on the phone itself** (no server, no accounts, no running costs). It already did everything in the brief. This is a recurring, deliberate strategy: **get something real and usable in your hands first**, then let actual use reveal what's next.

How it grew — and why each step was added

STEP	WHAT WAS ADDED	WHY (THE TRIGGER)
1	Core planner: tasks, dates, people, postpone, stats, recurring chores, “take turns”	The original brief + obvious quality-of-life features for two people.
2	Installable app + offline (PWA)	“We want to use it on our iPhones” — so it had to feel like an app and survive no signal.
3	Real-time sync via Supabase	Two phones each had their own copy. Manual export/import was clunky; you asked for live sync.
4	iPhone calendar integration	Your partner wanted tasks to appear in the Calendar app.
5	Push notifications (morning summary + completions)	You asked for reminders and “partner did a task” pings.
6	Full visual redesign (the “summer-mono” theme)	Once it worked, you wanted it to look clean and intentional.
7	Small refinements: task notes on the list, form fixes, more repeat intervals	Real daily use surfaced small friction — the best source of improvements.

KEY IDEA — ITERATIVE DEVELOPMENT

Each step was **working and shippable** before the next began. This is the single most important professional habit: *always keep the app in a working state*. Big-bang rewrites where “nothing works until everything works” are how projects die.

Why this order matters

Look again at the table: **function came before polish**. The design overhaul (step 6) happened only after sync and notifications worked. That's on purpose — a beautiful app that doesn't do the job is worthless, while a plain app that does the job is already valuable. Design is easy to change later (it's mostly colours and spacing); data models and sync are hard to change later. So the risky, structural things were tackled while the app was still small.

TRY IT

Open your repository's commit history on GitHub (the “commits” link). Each entry is one of these steps or a piece of one. Reading commit messages top-to-bottom is like reading the diary of how the app was built.

The three languages of the web

Every web app, including Sparkle, is built from three languages that do three different jobs. Knowing which is which is half of reading code.

LANGUAGE	JOB	ANALOGY
HTML	The <i>structure</i> and content — what elements exist (headings, buttons, lists)	The skeleton & organs
CSS	The <i>appearance</i> — colours, spacing, fonts, layout	The skin & clothing
JavaScript	The <i>behaviour</i> — what happens when you tap, calculate, save, sync	The muscles & brain

Sparkle is unusual (and beginner-friendly) in that all three live in a **single file**, `index.html`. Larger projects split them into many files, but the ideas are identical. Here's each one, shown with a real slice of your app.

HTML — elements and attributes

```
<button class="add-btn" onclick="openTaskForm()">+</button>
```

Read it like this: this is a `<button>` *element*. It has two *attributes* — `class="add-btn"` (a label CSS can hook onto for styling) and `onclick="openTaskForm()"` (when tapped, run the JavaScript function `openTaskForm`). The `+` between the tags is what you see on screen. Almost all HTML is this pattern: *an element, some attributes, some content*.

CSS — selectors and properties

```
.add-btn {  
  background: var(--accent); /* fill colour (graphite) */  
  color: var(--accent-ink); /* the "+" colour */  
  border-radius: 50%; /* make it a circle */  
}
```

The `.add-btn` is a *selector* — “find every element with class `add-btn`.” Inside the braces are *properties* (appearance settings). The `var(--accent)` bits are **variables**: colours defined once at the top of the file and reused everywhere, so the whole app can be re-themed by changing a handful of lines. That's exactly how your summer redesign was done.

JavaScript — the behaviour

```
function openTaskForm(id) {  
  // build the form, show it in a sliding sheet...  
}
```


JavaScript is where decisions and actions happen: reading what you typed, calculating a next due date, saving data, talking to Supabase. When you want to understand *what the app does* (as opposed to how it looks), you're reading JavaScript.

KEY IDEA

A quick triage when scrolling code: **angle-brackets** `<...>` = HTML (structure); curly-brace style blocks with **colour / padding** = CSS (looks); **function , if , = , (...)** = JavaScript (behaviour).

Anatomy of the app: reading `index.html`

The whole app is one file of roughly 1,600 lines. That sounds like a lot, but it's organised into clear regions. Once you know the regions, you can jump straight to the part you care about.

The file, top to bottom

REGION	WHAT'S THERE	LANGUAGE
The <code><style></code> block	All the colours, spacing and layout — including the summer theme and dark mode	CSS
The <code><body></code> markup	The skeleton of the screens: the setup screen, the app shell, the tab bar, the sliding “sheet”	HTML
State & constants	The data model, the list of icons, the repeat options	JavaScript
Helpers	Small reusable tools: date maths, escaping text, showing a toast message	JavaScript
Rendering	Functions that <i>draw</i> each screen from the data (<code>renderSchedule</code> , <code>renderStats</code> , ...)	JavaScript
Actions	What happens on tap: add, complete, postpone, edit, delete	JavaScript
Sync / notifications / calendar	Talking to Supabase	JavaScript

The one pattern that explains the whole app: *data* → *render*

This is the most important concept in the entire codebase, and it's beautifully simple. The app keeps **all** its information in a single object called `state`. Whenever something changes, it does two things: update `state`, then call `render()` to redraw the screen from that state. The screen is always just a *picture of the current data*.



Here is the actual render dispatcher from your app — notice how small it is:

```
function render() {
  if (activeTab === "schedule") renderSchedule();
  if (activeTab === "stats")    renderStats();
  if (activeTab === "settings") renderSettings();
}
```

That's the whole idea: “depending on which tab is open, draw that screen.” Each `render...` function builds a chunk of HTML text from `state` and drops it into the page. You never manually change one item on screen; you change the data and redraw. This makes bugs far rarer, because the screen can't drift out of sync with the truth.

KEY IDEA

If you remember one thing about the code: **the screen is a function of the data**. Find where the data changes, and you've found the behaviour. The drawing takes care of itself.

How a screen is drawn — a real example

This is (lightly trimmed) how one task row is turned into HTML:

```
function taskRow(t) {
  return `
    <button class="task-row" onclick="openTaskActions('${t.id}')">
      <div class="task-icon ${t.person}">${iconSvg(t.icon)}</div>
      <div class="task-main">
        <div class="task-title">${esc(t.title)}</div>
        <div class="task-sub">${esc(pname(t.person))} ... </div>
      </div>
      <div class="task-date">${humanDate(t.date)}</div>
    </button>`;
}
```

Even without knowing JavaScript, you can read the intent: build a button; inside it put an icon, the title, who it's assigned to, and the date. The `${...}` slots fill in values from the task `t`. When tapped, it calls `openTaskActions` with this task's `id`. This is representative of the entire app — screens are just text templates with values slotted in.

WATCH OUT — `ESC()`

See `esc(t.title)` rather than plain `t.title`? That `esc` function **escapes** the text so that if someone typed `<script>` as a task name, it's shown as harmless text instead of being run as code. Building HTML from user input *without* escaping is one of the most common security bugs on the web (called “XSS”). Spotting the presence or *absence* of escaping is a real code-reading skill — you now have it.

Data & state: where the information lives

All the app's memory lives in one object. Understand its shape and you understand what the app knows.

The `state` object

Here is the shape of Sparkle's entire memory:

```
state = {
  version: 2,
  people: { p1: { name: "Leanne" }, p2: { name: "Xiaoman" } },
  tasks: [ /* the planned, not-yet-done chores */ ],
  history: [ /* every completed chore, forever */ ],
  postponeLog: [ /* every time a task was moved */ ],
  tombs: { tasks: {}, history: {} }, // "tombstones" — see Chapter 11
  peopleMod: 0                      // when the names last changed
}
```

Everything the app shows is derived from this. The schedule is drawn from `tasks`; the stats are computed from `history` and `postponeLog`; the names come from `people`. A single task looks like this:

```
{
  id: "alb2c3",           // a unique random label
  title: "Vacuum living room",
  icon: "🧹",             // stored as an emoji (drawn as a line icon)
  person: "p1",           // assigned to person 1
  date: "2026-07-22",     // planned date (YYYY-MM-DD)
  firstDate: "2026-07-22", // the ORIGINAL planned date (never changes)
  postpones: 0,           // how many times it's been moved
  repeat: "1w",           // "every week"
  alternate: true,        // take turns each time
  notes: "",
  mod: 1721600000000      // when this task last changed (for sync)
}
```

KEY IDEA — THE DESIGN OF DATA IS THE DESIGN OF THE APP

Notice `firstDate` is kept *separately* from `date`. That single decision is what makes the “done on the first planned date” statistic possible: `date` moves when you postpone, but `firstDate` remembers where it started.

Good features usually trace back to a good data shape. When you read code, the data model tells you what's possible.

Where does this data physically live?

On the phone, in a browser feature called `localStorage` — a small, private box of text storage that survives closing the app. Saving is genuinely this simple:

```
function save() {  
  localStorage.setItem("sparkle.v1", JSON.stringify(state));  
  schedulePush(); // also queue a sync to Supabase (Chapter 11)  
}
```

`JSON.stringify` turns the `state` object into a single string of text so it can be stored; on the next launch the app reads it back and turns it into an object again. **JSON** (JavaScript Object Notation) is the universal text format for shuttling structured data around — you'll see it everywhere: storage, the database, the network.

WATCH OUT

`localStorage` is *per device*. That's the whole reason two phones don't automatically agree, and why Supabase (Chapter 10) was added. It also means clearing the app's storage — e.g. deleting and reinstalling the Home-Screen app — wipes the local copy. Your data is safe only because it's also mirrored in Supabase.

TRY IT

On a computer, open the app link in Chrome, press F12 to open developer tools, go to *Application* → *Local Storage*, and click the `sparkle.v1` entry. You'll see your entire app state as JSON text — the exact object from this chapter.

Following one action through the code

The best way to learn to read code is to trace a single action from tap to result. Let's follow “Mark as done” end to end. Every other action in the app follows the same shape.

Step 0 — the tap

In the task action sheet there's a button whose HTML is essentially:

```
<button onclick="openComplete('${t.id}')">Mark as done</button>
```

Tapping it runs `openComplete` with the task's id, which shows a little “who did it / on what date” confirmation. Pressing the final confirm runs `confirmComplete()` — the heart of the action.

Step 1 — read the confirmation, build a history entry

```
const entry = {
  id: uid(), title: t.title, icon: t.icon, person,
  firstDate: t.firstDate, plannedDate: t.date, completedDate,
  postpones: t.postpones,
  onFirst: t.postpones === 0 && completedDate <= t.firstDate,
  // ↑ “done on time” = never postponed AND finished by the original date
};
```

Read the `onFirst` line slowly — it's a small piece of real logic you can fully understand: a task counts as “done on the first planned date” only if it was *never postponed* (`postpones === 0`) and was completed on or before its original date. This one boolean feeds the “% on first date” statistic.

Step 2 — remove the task, remember it's gone

```
state.tasks = state.tasks.filter(x => x.id !== t.id);
state.tombs.tasks[t.id] = Date.now(); // a “tombstone” (Chapter 11)
```

`filter` makes a new task list containing every task *except* this one — that's how the completed chore leaves the schedule. The tombstone records “this id was deliberately deleted at this time,” which matters for sync so the task doesn't come back to life on the other phone.

Step 3 — if it repeats, plan the next one

```

if (t.repeat) {
  let next = applyRepeat(t.date, t.repeat);           // e.g. +1 week
  const nt = { ...t, id: uid(), date: next, firstDate: next, postpones: 0,
               person: t.alternate ? other(person) : t.person };
  state.tasks.push(nt);    // add the next occurrence
}

```

This is the “recurring chore” feature. `{ ...t, ... }` means “make a copy of the task, then override these fields.” If `alternate` is on, `other(person)` flips `p1 ↔ p2` so you take turns. This is why completing a weekly chore instantly plans next week’s, assigned to the other person.

Step 4 — record, save, redraw, notify

```

state.history.push(entry);
save();                      // persist locally + queue sync
render();                   // redraw the schedule
notifyPartnerCompleted(entry); // ping the other phone (Chapter 12)

```

KEY IDEA — EVERY ACTION HAS THE SAME SKELETON

(1) read input → (2) change `state` → (3) `save()` → (4) `render()`. Postpone, edit, delete, add — all follow it. Once you’ve traced one, you can trace them all. Try reading `doPostpone` next; you’ll recognise every step.

Making it an installable app (PWA, offline, updates)

A plain web page can't be added to your Home Screen or work offline. Three small pieces turn Sparkle into a **Progressive Web App (PWA)** — the technology that lets a website behave like an installed app.

1 · The manifest — “I am an app”

`manifest.webmanifest` is a tiny file that tells iOS the app's name, icon, colours, and that it should open full-screen without browser chrome. It's why “Add to Home Screen” gives you a proper icon and a full-screen experience rather than a Safari tab.

2 · The service worker — offline & updates

A **service worker** (`sw.js`) is a small script the browser keeps running in the background, even when the app is closed. It sits between the app and the network and can answer requests from a local cache. That's what makes Sparkle open instantly and work with no signal.

```
// simplified idea of the service worker
const CACHE = "sparkle-v5"; // the version tag
// On install: save the app's files into the cache.
// On each request: try the network, fall back to the cache when offline.
```

KEY IDEA — CACHE VERSIONING

See `sparkle-v5`? Every time the app changed meaningfully, that number was bumped (v1 → v5). Changing the name makes the browser throw away the old cached files and fetch the new ones. This is the classic fix for “I updated the app but my phone still shows the old version.” Recognising a *version tag* in caching code is a useful pattern to know.

3 · Auto-updating

Installed web apps are notorious for clinging to old versions. Late in the project we added a few lines so that when a newer version is detected, the app reloads itself once — so future updates just appear. This is exactly the sort of small, targeted fix that comes from real-world use (Chapter 3's step 7).

WATCH OUT — THE HOME-SCREEN ICON

One thing a service worker *cannot* refresh is the Home-Screen icon: iOS snapshots it when you install. Changing the icon requires removing and re-adding the app. This isn't a bug — it's an Apple limitation, and knowing where the platform's edges are is part of understanding the system.

Git & GitHub: version control and hosting

GitHub plays two separate roles in this project, and beginners often blur them: it's a **history keeper** (version control) and it's a **web host** (GitHub Pages). Let's separate them.

Git — a time machine for code

Git is a tool that records snapshots of your files over time. Each snapshot is a **commit** — a saved point with a message describing what changed. You can look back at any commit, compare versions, and undo. GitHub is the website that stores those commits in the cloud and lets you view them.

TERM	MEANING	IN YOUR PROJECT
Repository (“repo”)	The project folder, with its full history	<code>Barcode-Bleeb/cleaning-scheduling</code>
Commit	One saved snapshot + a message	e.g. “Add push notifications...”, “Redesign UI...”
Branch	A parallel line of work	<code>claude/cleaning-schedule-app-9hszwq</code>
Push	Upload local commits to GitHub	What happened after every change we made

KEY IDEA — WHY COMMIT MESSAGES MATTER

Each commit in your repo has a clear message explaining *what* and *why*. Read top to bottom, they are the story of the app. Good messages are a gift to your future self: when something breaks, you can find exactly which change caused it. This is why professionals commit in small, described steps rather than one giant “stuff” commit.

GitHub Pages — free hosting

A repository is just files; for a phone to open the app, those files must be served on the internet. **GitHub Pages** does that for free: you flip a setting pointing it at your branch, and GitHub publishes the files at a public address like `https://barcode-bleeb.github.io/cleaning-scheduling/`. Every time we pushed a commit, Pages re-published within a minute or two. That's the entire “deployment” process for this app.

THE DEPLOY FLOW

Edit code

change `index.html`



Commit

snapshot + message



Push

upload to GitHub



Pages publishes

live on the web in ~1 min

KEY IDEA — THE TWO HATS

Git/GitHub = history. GitHub Pages = hosting. They happen to be the same company, but they're different jobs. A repo can exist without Pages; Pages is an optional feature you switch on.

REMEMBER THIS FROM OUR PROJECT

Early on, pushing failed with a “403” error. The cause wasn't the code — it was that the Claude GitHub App hadn't been *installed on the repository* with write permission yet. This is a good lesson: not every error is a code error. Permissions, hosting settings, and access are a real and separate category of problem to check.

Supabase in depth: the backend that ties two phones together

This is the chapter you asked me to go deepest on. Supabase is where Sparkle stopped being a solo app on one phone and became a shared app for two. Take your time here — the ideas transfer to almost every app you'll ever meet.

What problem does Supabase solve?

Recall from Chapter 6 that each phone stores its data in its own `localStorage`. Two phones, two separate boxes, no connection. If Leanne completes a chore, Xiaoman's phone has no way to know. To share, both phones need to talk to **one thing in the middle** that holds the real, shared data. That middle thing is a **backend**, and Supabase is the backend we used.

What is Supabase?

Supabase is a “**Backend-as-a-Service**” (**BaaS**): it gives you the common backend pieces — a database, an access layer, live updates, and small server functions — ready-made, so you don't have to build and run your own server. Think of it as renting a professionally-run engine room instead of building one. For a two-person app it's free, and there's nothing to maintain.

SUPABASE PIECE	WHAT IT IS	HOW SPARKLE USES IT
Postgres database	A reliable, industry-standard place to store data in tables	Holds the shared household data, and the notification subscriptions
Auto REST/JS access	A ready-made way for the app to read/write the database over the internet	The app reads & writes the household row
Realtime	Live “something changed” push updates	The instant one phone saves, the other is notified
Edge Functions	Small server programs you deploy	The calendar feed and the push-notification sender
Row Level Security	Database rules about who may read/write what	Guards the tables (see the security note)

The database, concretely: tables and rows

A **database** is organised into **tables** (like spreadsheets). Each table has **columns** (fields) and **rows** (records). Sparkle's main table is `sparkle_state`, and it's deliberately tiny — *one row per household*:

```
create table sparkle_state (
  household text primary key, -- a random id shared by your two phones
  doc        jsonb not null,  -- your ENTIRE app data, as JSON
  rev        bigint not null, -- a version counter (see Chapter 11)
  updated_at timestampz
);
```

KEY IDEA — THE CLEVER SIMPLIFICATION

Instead of many tables (one for tasks, one for history, ...), Sparkle stores the *whole* app state as a single JSON value in one `doc` column. This is unusual for big apps but perfect for a two-person one: it keeps the backend trivial and reuses the exact same data shape the app already uses locally. **Choosing the simplest structure that fits the problem is a mark of good engineering, not a shortcut.**

How the app talks to Supabase

The app includes Supabase's small JavaScript library and creates a “client” with two values you copied from your Supabase dashboard: the **Project URL** (the address of your backend) and the **anon key** (a public access key). Then reading and writing is about this simple:

```
// read the household's row
const { data } = await client
  .from("sparkle_state")
  .select("doc, rev")
  .eq("household", householdId)
  .maybeSingle();
```

Read it as a sentence: “from the `sparkle_state` table, select the `doc` and `rev` columns, where `household` equals our id, and expect at most one row.” This style — a chain of `.from().select().eq()` — is Supabase's query builder, and it reads almost like English once you've seen it a few times.

Two keys, and why the difference is the most important security lesson here

KEY	WHO HAS IT	POWER
anon (public) key	Shipped inside the app — effectively public	Only what your security <i>rules</i> allow
service role key	Kept secret, only on the server (Edge Functions)	Full access — bypasses all rules

WATCH OUT — THE ANON KEY IS PUBLIC ON PURPOSE

Because the app runs on the phone, *anything* it contains can be read by a determined user (Chapter 2). So the anon key is designed to be public — it is **not** a password. What actually protects your data is **Row Level Security (RLS)**: rules stored in the database itself that decide what any given key is allowed to do. The service-role key, which *can* do anything, is never put in the app — it lives only inside Edge Functions on the server. **Never put a secret key in frontend code.** This is one of the most important habits in all of web development.

What protects a Sparkle household, honestly

Your data's practical protection is that the *household id* in your pairing code is a long random value that only your two phones know. Anyone without it can't find your row. For a private two-person cleaning list this is a sensible, proportionate level of security — and, importantly, you now understand *why* it's the household id and not the anon key doing the protecting.

Edge Functions — a little bit of “real server”

Some jobs can't be done from the phone: sending a push notification requires a secret signing key, and the iPhone Calendar app needs a plain web address to fetch. For these, Supabase runs small server programs called **Edge Functions** (in the `supabase/functions/` folder of your repo):

- `calendar` — builds a calendar feed of your tasks that iOS can subscribe to.
- `push` — sends the morning summary and the “partner completed a task” notifications, using the secret key that must never touch the phone.

A scheduled job (`pg_cron`, a database timer) calls the `push` function every morning to send the daily summary — a nice example of the backend doing work even when neither phone is open.

TAKEAWAYS FROM THE SUPABASE CONNECTION

1. A backend exists to be the shared source of truth that phones can't be for each other.
2. Frontend code is public; secrets and “full-power” keys live only on the server.
3. Security rules (RLS) live *in the database*, not in the app — the app is never trusted.
4. The simplest data structure that fits (here, one JSON row per household) is often the best one.
5. Some tasks are inherently server-side (secret keys, scheduled jobs) — that's what Edge Functions are for.

How real-time sync actually works

Sync sounds like magic, but it's a handful of understandable rules. Getting it right is genuinely hard — this is the most sophisticated part of your app — so this chapter also shows you what “careful code” looks like.

The hard problem

Two phones, each with their own copy, possibly *both* offline and both making changes. When they reconnect, whose version wins? If you naively let “the last one to save” overwrite everything, you'd lose changes. Sparkle instead **merges** the two copies intelligently so nobody's work is lost.

The four rules of the merge

SITUATION	RULE	WHY
Same task edited on both phones	Newest change wins (compare each task's <code>mod</code> timestamp)	The later edit is presumably the intended one
Different tasks added on each phone	Keep both	They don't conflict — union them
Completed history & postpone log	Combine, never delete (append-only)	History is a permanent record
A task was deleted/completed on one phone	Tombstone wins — it stays gone	Otherwise the other phone's stale copy would resurrect it

KEY IDEA — TOMBSTONES

When you delete or complete a task, the app doesn't just remove it; it records a **tombstone** — “id X was removed at time T.” Without this, sync would see the task still present on the other phone and happily add it back. Tombstones are how distributed systems remember that something was *intentionally* gone. (There's even an “undo” subtlety: if you undo a completion, the tombstone is cleared so the task legitimately returns.)

Avoiding lost updates — the version counter

Remember the `rev` column? Each save writes “update the row to the new data, *but only if* `rev` is still the number I last saw, and bump it by one.” If in the meantime the other phone saved (changing `rev`), your update is politely rejected; the app then re-reads, merges, and retries. This technique — **optimistic concurrency** — prevents two simultaneous saves from silently clobbering each other.

```
// update only if nobody else changed the row first
.update({ doc, rev: currentRev + 1 })
.eq("household", id)
.eq("rev", currentRev) // ← the safety check
```

Offline-first

The app *always* saves locally first and shows the change immediately; syncing to Supabase happens in the background and retries when the connection returns. This is called **offline-first**, and it's why Sparkle never feels like it's “waiting for the internet.”

WATCH OUT — THIS IS WHERE BUGS LOVE TO HIDE

Sync, merging, and “what happens offline” are famously the hardest things to get right in app development. This is precisely why we wrote automated tests that simulate *two phones* editing at once and check they end up identical (Chapter 13). When you see complex code like this, the right instinct isn't “trust it” — it's “how is this *tested*?”

Notifications & calendar feeds

Two features that reach outside the app and into the phone's system: notifications and the Calendar app. Both lean on the backend, and both teach a useful lesson about platform rules.

Push notifications

Getting a banner on your phone when your partner finishes a chore involves a surprising number of players, but the concept is simple: your phone *subscribes* to notifications and gets a private “address.” When something happens, the `push` Edge Function sends a message to that address, and Apple's servers deliver it to the phone.

- **VAPID / Web Push** — the standard for sending these messages. It uses a secret signing key so only *your* server can send to your users. That key lives only in the Edge Function — never on the phone.
- The morning summary is triggered by a **scheduled database job**, not by either phone.
- Completion pings are triggered when a task is marked done, and deliberately skip the phone that did it.

WHY THIS ONLY WORKS INSTALLED

Apple only allows web-push for apps *added to the Home Screen* (iOS 16.4+), not for pages in a Safari tab. That's a platform rule, not a limitation of your app — the same “know where the platform's edges are” lesson from Chapter 8.

The calendar feed

The Calendar integration uses an old, robust internet standard called **iCalendar** (`.ics`) — the same format every calendar app understands. The `calendar` Edge Function generates a live `.ics` feed of your planned tasks; you subscribe to it once in iOS, and the calendar refreshes itself. No custom Apple integration required — just a well-chosen standard.

KEY IDEA — PREFER STANDARDS

Both features work by speaking an existing *standard* (Web Push, iCalendar) rather than inventing something new. Reaching for a well-supported standard instead of a bespoke solution is almost always the wiser, more durable choice.

Testing: how we proved it works

*“It seems to work” is the enemy. Throughout this project, every non-trivial change was checked by an **automated test** — a program that drives the app like a user and verifies the result. This is the strongest antidote to vibe coding there is.*

What an automated test actually is

A test is just code that *uses* your code and then **asserts** that the outcome is what it should be. For Sparkle we used a tool called **Playwright**, which launches a real (invisible) browser, opens the app, clicks buttons, and reads the screen — exactly as a person would, but instantly and repeatably.

```
// plain-English version of a real Sparkle test
open the app
add a task called "Sync test task"
mark it done, say Xiaoman did it
assert it moved from the schedule into history
assert the "done on first date" flag is correct
```

What we tested, and why it mattered

TEST SUITE	WHAT IT PROVES
End-to-end	Adding, completing, postponing, undo, recurring & “take turns” all work
Sync	Two simulated phones editing at once end up <i>identical</i> — the hard case from Chapter 11
Notifications	Enabling, toggling, and completion pings send exactly the right requests
Web-push encryption	Our from-scratch encryption matches trusted reference libraries byte-for-byte

KEY IDEA — THE REAL VALUE OF TESTS

Tests do two things. First, they catch mistakes *before* they reach your phone. Second, and just as important, they let you **change the app fearlessly**: after the big redesign, re-running the tests proved that none of the behaviour had broken, even though almost every screen looked different. Tests are what make an app safe to keep improving.

TRY IT — THE HONESTY CHECK

When you (or an AI) add a feature, ask a simple question: “*How do we know it works?*” If the answer is “I clicked it once,” that’s vibe coding. If the answer is “here’s a test that proves it, and it passes,” that’s engineering. You’re allowed to ask for the test.

A little design literacy

You have strong instincts about how the app should look. A few principles turn instinct into vocabulary — useful both for judging design and for asking for what you want.

Colour with meaning

In Sparkle, colour is *rationed* and each colour *means* something: yellow is Leanne, purple is Xiaoman, red means “overdue,” graphite is for buttons. Everything else is near-white and grey. That discipline — most of the screen neutral, colour reserved for meaning — is why the app feels calm rather than busy. When colour is decoration, it's noise; when colour is information, it's design.

The pastel problem — a real lesson from your redesign

You asked for “summer light yellow.” A pale yellow dot on a near-white background is nearly invisible — too little contrast. The fix was to give each person *two* shades: a pale wash for large areas (the round icon backgrounds) and a deeper tone for small marks (dots, chart bars). This is **contrast** at work: the smaller or more important a thing, the more contrast it needs to be seen.

KEY IDEA — CONTRAST IS LEGIBILITY

Whenever text or a small element is hard to read, the culprit is almost always insufficient contrast with its background. This isn't taste — it's measurable, and it's also an accessibility requirement. “Make it lighter/softer” and “keep it readable” are in tension, and good design resolves that tension deliberately.

Hierarchy and whitespace

The redesign leaned on **typographic hierarchy** (big thin numbers, medium titles, tiny uppercase labels) and **whitespace** (empty space as a separator) instead of boxes and borders. Notice how the stats screen guides your eye purely with size and spacing. Empty space isn't wasted space — it's what makes the important things stand out.

Reading code like a reviewer: a beginner's rubric

This is the chapter that most directly serves your goal: a practical checklist for looking at a piece of code and forming a view — “this looks solid” or “this might be trouble” — without needing to understand every line.

How to read a function you've never seen

1. **Read the name first.** A good name (`confirmComplete` , `doPostpone` , `mergeDocs`) tells you the intent before you read a single line. Vague names (`doStuff` , `handle2`) are themselves a mild warning sign.
2. **Find the inputs and the output.** What does it take in (the parts in the `(...)`)? What does it produce or change? This frames everything else.
3. **Look for the “happy path.”** Ignore edge cases at first and read the main story: the normal sequence of steps.
4. **Then look for the guards.** The `if (!t) return;` lines at the top are the function protecting itself against bad input. Their *presence* is a good sign.

Good signs (green flags)

WHEN YOU SEE...	IT SUGGESTS...
Clear, descriptive names	The author understood the problem
<code>if (!something) return;</code> near the top	Defensive coding — handles missing/empty input
<code>esc(...)</code> around anything a user typed, before putting it on screen	Protection against the XSS security bug
A <code>try { ... } catch { ... }</code> around network calls	The code plans for the internet failing
Small functions that each do one thing	Easier to understand, test, and fix
A comment explaining <i>why</i> (not what)	The author flagged something subtle for future readers

Warning signs (red flags)

WHEN YOU SEE...	BE SUSPICIOUS BECAUSE...
User input put straight into the page with no <code>esc</code>	Possible security hole (XSS)
A network/database call with no error handling	The app may break or freeze when offline
The same block of code copy-pasted many times	A fix must be made in many places — bugs breed here
Mysterious numbers with no name or comment (“magic numbers”)	Nobody will remember what <code>86400</code> meant (it's seconds in a day)
A function that's hundreds of lines long	Probably doing too many things; hard to reason about
“It works” with no test for something tricky	Untested complex code is where bugs live

KEY IDEA — YOU DON'T JUDGE CORRECTNESS, YOU JUDGE CARE

As a beginner you usually can't verify that logic is 100% correct. You *can* assess whether the code was written **carefully**: does it name things well, guard its inputs, escape user text, handle failure, avoid needless repetition, and get tested? Careful code isn't always correct — but careless code is almost never trustworthy. Judging care is a realistic, valuable skill you already have now.

TRY IT ON YOUR OWN APP

Open `index.html` and find the function `doImport` (it reads a pasted sync code). See the `try { ... } catch { ... }` and the check that the pasted data has the right shape before trusting it? That's defensive coding. Now find `saveTaskForm` and notice it refuses to save a task with no title. You just reviewed code.

Glossary

Plain-language definitions for every important term in this manual. Skim it now; return to it whenever a word stops you.

TERM	MEANING
Frontend / client	The part of the app that runs on the user's device (the browser).
Backend / server	The part that runs in the cloud; the shared source of truth.
HTML / CSS / JavaScript	The three web languages: structure / appearance / behaviour.
Element / attribute	An HTML building block (<code><button></code>) and its settings (<code>class="..."</code>).
Function	A named, reusable block of instructions.
State	The object holding all the app's current data.
Render	To draw the screen from the current state.
JSON	A universal text format for structured data.
localStorage	A small per-device text store in the browser that survives restarts.
PWA	Progressive Web App — a web app installable like a native one.
Service worker	A background script enabling offline use and updates.
Cache	A local copy of files kept for speed and offline use.
Git / commit / branch / push	Version control: history tool / a snapshot / a line of work / uploading.
Repository (repo)	A project plus its full history.
GitHub Pages	Free web hosting that publishes a repo's files.
Deploy	To publish a new version so users can get it.
Database / table / row / column	Structured storage / a spreadsheet-like set / one record / one field.
Postgres	The specific, industry-standard database Supabase uses.
BaaS	Backend-as-a-Service — ready-made backend pieces (Supabase).
anon key	A public access key shipped in the app; not a password.
service role key	A full-power secret key kept only on the server.
Row Level Security (RLS)	Database rules controlling who may read/write which rows.
Edge Function	A small server program you deploy (calendar, push).
Realtime	Live “something changed” updates pushed to the app.
Optimistic concurrency	“Save only if nobody changed it first” — prevents lost updates.
Tombstone	A record that an item was intentionally deleted, so sync won't revive it.
Offline-first	Save locally & instantly; sync in the background.
XSS / escaping	A security bug from unescaped input / the fix that neutralises it.

MVP	Minimum Viable Product — the simplest version that delivers the core value.
Assertion / test	A check that the outcome is as expected / code that runs those checks.

Where to go next

You've gone from “I described an app” to “I can read the app and reason about it.” Here's how to keep that momentum.

Small, safe experiments

- **Read one commit a day.** Open your GitHub commit history and read the message + the changed lines of a single commit. Ten minutes; enormously clarifying.
- **Change one colour.** Find a `--` variable at the top of `index.html`'s style block, change its value, and see what moves. Nothing you do here can break the data.
- **Trace one more action.** You traced “Mark as done” in Chapter 7. Now trace `doPostpone` or `saveTaskForm` and confirm the same four-step skeleton.
- **Break something on purpose (locally).** In your browser's dev tools you can experiment freely; a refresh restores everything from storage.

Questions worth asking on any future feature

- “Is this frontend or backend?”
- “Where does this data live, and who else needs it?”
- “What happens if the internet is off, or the input is empty?”
- “How do we know it works — is there a test?”
- “Is any secret ending up in code that ships to the phone?”

THE ONE HABIT THAT MATTERS MOST

Keep the app **working at every step**, make **small changes**, and **verify** each one. That single discipline — more than any language or framework — is what separates people who build reliable software from people who merely generate it. You've now seen it done from start to finish. That's a real foundation.

You built a genuinely useful app, connected it to a real backend, gave it notifications and a calendar feed, redesigned it, and now you can read it. That's a lot for a first project. Well done — and enjoy the tidy house. 🧹✨